

26/07/2026

Assignment - 1

- 1) Explain the different Data Types available in Python. Differentiate list, tuple, set and dictionary.

→ A data type specifies the type of value that a variable can store. Python provides several built-in data types to handle different kinds of data.

Types of Data Types in Python:

(1) Numeric Data Type: It is used for storing numerical values.

→ int - integer numbers

→ float - decimal numbers

→ complex - complex numbers

Example: $a = 20$

$b = 15.75$

$c = 3 + 4j$

(2) String Data Type: Stores a sequence of characters enclosed in single, double, or triple quotes.

Example: `name = "Python"`

(3) List: A list is an ordered, mutable collection that allows duplicate values.

Example: `marks = [85, 90, 78]`

(4) Tuple: A tuple is an ordered but immutable collection.

Example: `colors = ("Red", "Blue", "Green")`

(5) Set: A set is an unordered collection that stores unique elements.

Example: `numbers = {1, 2, 3, 4}`

(6) Dictionary: A dictionary stores data in key-value pairs.

Example: student = {
 "Name": "Raj",
 "Age": 20
 }

→ Feature	List	Tuple	Set	Dictionary
Ordered	Yes	Yes	No	Yes
Mutable	Yes	No	Yes	Yes
Duplicate Values	Allowed	Allowed	Not Allowed	Keys Not Allowed
Indexing	Yes	Yes	No	Access by Key
Symbol	[]	()	{ }	{key: value}

→ Python provides multiple data types to efficiently store and manipulate different kinds of data. Selecting the appropriate data types improves program performance and readability.

2) Discuss the role of indentation in Python.

↳ Indentation refers to the spaces or tabs used at the beginning of a line of code. In Python, indentation defines the scope of loops, functions, conditions, and classes.

Unlike many programming languages, Python does not use braces { } to define blocks.

→ Importance of Indentation:

(1) Defines Code Blocks: Indentation groups statements belonging to the same block.

Example: `if x > 10:`

`print("Greater")`

(2) Improves Readability: Proper indentation makes programs easier to understand.

(3) Mandatory in Python: Incorrect indentation results in an error.

Example: `if x > 5:`

`print(x)`

Output:

IndentationError

(4) Used in Loops:

Example: `for i in range(5):`

`print(i)`

(5) Used in Functions:

Example: `def greet():`

`print("Welcome")`

→ Advantages:

- Makes code neat and organized.
- Eliminates unnecessary braces.
- Improves maintainability.
- Prevents logical errors.
- Encourages clean programming.

→ Indentation is one of Python's most important Features. Without proper indentation, a Python program cannot execute successfully.

3) Explain the range() function of Python with a suitable example.

→ The range() function generates a sequence of numbers. It is mainly used with loops.

Syntax: range(start, stop, step)

Parameters:

- start - Starting number (default 0)
- stop - Ending number (not included)
- step - Difference between numbers (default 1)

→ Forms of range():

(1) One Parameter

Code:

```
for i in range(5):  
    print(i)
```

Output:

0
1
2
3
4

(2) Two Parameters

Code:

```
for i in range(2, 7):  
    print(i)
```

Output:

2

3

4

5

6

(3) Three Parameters

Code:

```
for i in range(2, 11, 2):  
    print(i)
```

Output:

2

4

6

8

10

→

Advantages:

- Saves memory.
- Generates number efficiently.
- Simplifies loops.
- Supports positive and negative steps.

→ The range() function is widely used for iteration and generating sequences in Python.

4) Write a Python program to display the Fibonacci series. The number of terms is to be inputted by the user.

↳ Program : Fibonacci Series

Code :

```
n = int(input("Enter number of terms: "))
```

```
a = 0
```

```
b = 1
```

```
print("Fibonacci Series: ")
```

```
for i in range(n):
```

```
    print(a, end=" ")
```

```
    c = a + b
```

```
    a = b
```

```
    b = c
```

Output :

Enter number of terms: 8

Fibonacci Series:

0 1 1 2 3 5 8 13

→ The Fibonacci series is generated by adding the previous two numbers, making it an excellent example of loops and variables.

- 5) Write a Python program to perform the following operations about functions: 1. Function with multiple return values. 2. Default arguments.

→ (1) Function with Multiple Return Values:

Code:

```
def calculate(a,b):  
    return a+b, a-b, a*b
```

```
sum, difference, product = calculate(10,5)
```

```
print("Sum=", sum)
```

```
print("Difference =", difference)
```

```
print("Product =", product)
```

Output:

Sum = 15

Difference = 5

Product = 50

→ (2) Function with Default Arguments:

Code:

```
def greet(name="Student"):  
    print("Welcome", name)
```

```
greet()
```

```
greet("Rahul")
```

Output:

Welcome Student

Welcome Rahul

6) Explain looping statements in Python. Give examples of while and for loop.

↳ Loop: A loop repeatedly executes a block of code until a specified condition is satisfied.

→ Types of Loops:

(1) For Loop: Used when the number of iterations is known.

Syntax: `for variable in sequence:`
 statements

Example: `for i in range(1, 6):`
 `print(i)`

Output: 1

2

3

4

5

(2) while Loop: Used when the number of iterations is not known.

Syntax: `while condition:`
 statements

Example: `i = 1`

`while i <= 5:`

`print(i)`

`i += 1`

Output: 1

2

3

4

5

→

for Loop

while Loop

- Used for fixed iterations.

- Used for unknown iterations.

- Works on sequences.

- Depends on condition.

- Simpler syntax.

- Needs condition update.

→ Loops reduce repetitive coding and make programs efficient.

7) List and explain any three magic Functions used in Python/Jupyter.

↳ Magic Functions (Magic Commands) are special commands available Jupyter Notebook/ IPython that begin with % or %%. They simplify common tasks such as measuring execution time, running scripts, and listing available commands.

(1) %time : Measures the execution time of a single statement.

Example: `%time sum(range(100000))`

Use:

- Checks execution time of code.
- Helps optimize program performance.

(2) `%run`: Runs an external Python script inside Jupyter Notebook.

Example: `%run program.py`

Use:

- Executes Python files.
- Useful for testing programs.

(3) `%pwd`: Displays the current working directory.

Example: `%pwd`

Output: 'C:\\Users\\Student'

Use:

- Shows the current folder.
- Helps locate notebook files.

→ Advantages of Magic Functions:

- Increase productivity.
- Easy debugging.
- Performance measurement.
- Quick execution of scripts.
- Simplify notebook operations.

→ Magic Functions are powerful Jupyter features that make coding, testing, and performance analysis easier.

8) What are the advantages of Python as a preferred programming language for Data Science?

→ Python is one of the most popular programming languages for Data Science because it is simple, powerful, and supported by a vast ecosystem of

libraries and tools.

→ Advantages of Python in Data Science:

- (1) Easy to Learn: Python has simple syntax that is easy for beginners.
- (2) Rich Libraries: Provides powerful libraries such as - Numpy, Pandas, Matplotlib, Seaborn, Scikit-learn, TensorFlow, etc.
- (3) Data Visualization: Supports attractive graphs and charts using Matplotlib and Seaborn.
- (4) Machine Learning Support: Python includes libraries for machine learning, artificial intelligence, and deep learning.
- (5) Large Community Support: Millions of developers contribute tutorials, packages, and documentation.
- (6) Cross-Platform Compatibility: Python programs run on Windows, Linux, and macOS with minimal changes.
- (7) Integration with Big Data: Python integrates with Hadoop, Spark, SQL databases, and cloud platforms.
- (8) Open Source: Python is free to download and use without licensing costs.

→ Applications of Python in Data Science:

- Data Analysis
- Data Visualization
- Machine Learning
- Artificial Intelligence

- Business Analytics
- Predictive Modeling
- Deep Learning

→ Python is the preferred language for Data Science because of its simplicity, powerful libraries, excellent community support, and wide range of applications. It enables efficient data analysis, visualization, and machine learning, making it an ideal choice for students, researchers, and industry professionals.